ResolveDesk

01 Create, assign, prioritize and resolve support tickets.

02 React interface, FastAPI services and persistent PostgreSQL storage.

03 Verified locally and on shared AWS EKS. Temporary cloud resources were removed after recording.

A small workflow, implemented end to end

01 Requester email, subject, description, priority and assignee stay with each ticket.

02 The workflow is OPEN -> IN_PROGRESS -> RESOLVED; teams can reopen tickets.

03 Search and filters help triage; summary counts expose unresolved urgent work.

04 Assumption: an internal classroom demonstration with synthetic data and no authentication.

Separate responsibilities

01 React + Vite: ticket workspace, forms and accessible feedback.

02 FastAPI: HTTP routes; service functions: workflow operations; Pydantic: validation.

03 SQLAlchemy + PostgreSQL: storage. Alembic: explicit schema migrations.

04 Docker Compose: local integration. Helm + Kubernetes: repeatable deployment.

One application, two delivery paths

Browser -> **React / Nginx** -> **FastAPI** -> **PostgreSQL**

GitHub Actions -> GHCR -> ArgoCD / Helm -> EKS

Prometheus <- /metrics Grafana <- Prometheus

01 Local: Compose connects the same three application services.

02 AWS: Terraform creates the VPC and EKS; Helm defines the workload.

Test before publication

01 GitHub Actions runs backend tests, frontend tests and a production frontend build.

02 Build each image once and tag it with the source commit SHA.

03 Trivy blocks HIGH and CRITICAL findings and preserves scan reports.

04 Deploy scanned images to temporary Kind, smoke-test them, then publish to GHCR.

Make infrastructure reproducible

01 VPC spans two availability zones, with public and private subnets.

02 Temporary workers use public subnets with automatic public IPs. Security groups allow no public inbound traffic.

03 The account had no Elastic IP quota available, so the demo uses no NAT gateway.

04 The Kubernetes API is restricted to the operator IP. EBS CSI uses a dedicated IAM role.

05 Recovered apps ran successfully. Apply was interrupted during old worker-group deletion; final destroy succeeded.

Deploy and recover predictably

01 Frontend and backend Deployments use probes, resource requests and limits.

02 A database StatefulSet uses persistent storage; credentials come from a Secret.

03 A migration Job applies Alembic before the application is considered ready.

04 HPA adjusts backend replicas; an Ingress can provide a single application endpoint.

05 Verified two frontend and two backend replicas, persistent gp3 storage and valid HPA CPU measurements.

Git records the desired release

01 ArgoCD reads the Helm chart and release values from Git.

02 Commit-SHA image tags connect a deployed release to its tested source.

03 A values change updates the desired state; ArgoCD reconciles the cluster.

04 Recorded ArgoCD evidence shows the deployed application Synced and Healthy.

05 The backend replica field is ignored by ArgoCD so HPA can own scaling.

Observe behavior, not just containers

01 /health reports process liveness; /ready checks access to the migrated database.

02 /metrics exposes HTTP request counts, duration and status metrics.

03 A ServiceMonitor connects the API to Prometheus; verify its target is UP.

04 Grafana displays request rate, p95 latency and server errors.

05 Generate traffic and allow scrape intervals before recording the dashboard.

Evidence and lessons

01 26 backend tests and four frontend tests passed; CI built, scanned, smoke-tested and published both images.

02 Recorded EKS deployment: working ingress, ArgoCD Synced/Healthy, two backend monitoring targets UP and populated Grafana panels.

03 Terraform destroy removed 59 resources; all 11 cleanup checks passed. KMS deletion is scheduled for November 15, 2026.

04 Quota blocked the first NAT plan. Recovered apps ran, but apply was interrupted during old worker-group deletion.

05 Classroom demo: synthetic data, no authentication and one PostgreSQL instance. Production needs access control, backups and recovery testing.